Manual para la Corrección de Pruebas Técnicas

Revisión de Cambios:

Fecha	Autor	Cambios
17/12/21	Ismael Garrido	Inicial
31/01/22	Ismael Garrido	Detalle qué hay en la sección otros
10/05/22	Nicolás Caballero Patrick Silveira	Manual de corrección de pruebas Frontend

Introducción

Este documento se propone introducir al método de corrección de pruebas técnicas en Qualabs, dando una guía y ejemplo de cómo realizar la corrección. Forma parte del entrenamiento como corrector de pruebas técnicas.

Junto a este documento, hay un set de pruebas para corregir como entrenamiento. Además, hay un checklist donde se ponen los resultados de cada evaluación.

Se recomienda realizar una copia personal del checklist, y para cada corrección, duplicar la hoja y renombrarla con el nombre de la persona que envió la prueba.

Prueba técnica

Esta prueba cuenta con 2 o 3 partes dependiendo de si el candidato se postula como backend developer o como frontend developer. En el caso de que el candidato se postule como frontend developer se le agrega a la prueba un ejercicio de desarrollo web.

La parte A refiere a un ejercicio básico en donde se pide catalogar unos archivos json (los mismos son dados en un zip aparte). Este ejercicio evalúa que el candidato tenga habilidades básicas de programación realizando un ejercicio simple.

Por otra parte, el ejercicio B plantea un desafío de algoritmos. El mismo fue creado en base a un ejemplo real que sucedió en un proyecto con un cliente. A diferencia del primer ejercicio este intenta evaluar los conocimientos del candidato y su capacidad de resolver problemas más complejos.

El ejercicio C plantea el desafío de construir una web imitando un diseño dado. El mismo fue creado pensando en que al realizarse se pueda mostrar tanto habilidades de CSS, como conocimientos de reuso de componentes y manejo de datos.

Criterios de corrección

Se detalla a continuación una visión general de los aspectos que se evalúan. La lista completa está en el documento “checklist de corrección”. Haga una copia de este documento para su uso personal.

El checklist se rellena con 0 o 1 en la primera columna. Esto genera el texto final que luego debemos copiar en la tarea de corrección en Asana.

El checklist de corrección sigue el criterio de que si la prueba está toda bien, todo aparece con un check verde(✅). El objetivo de este código de colores es que visualmente sea rápido de entender. Por esto, aspectos negativos como “comentarios excesivos” aparecen como “no hay comentarios excesivos”. De esta manera, si no hay comentarios excesivos veremos todo en verde.

Ciertos errores se consideran muy graves y se debe catalogar la prueba como “no suficiente”. Estos errores son:

- Parte A con errores
- Parte B no consigue un resultado válido
- Parte C no se consigue un resultado estéticamente similar o hace todo en mismo componente.
- Hardcodear los proveedores que espera encontrar (authn.provider_1, authz.provider_1, etc)

En la checklist, están marcados con ❗ los errores graves, y con 😀 las cosas positivas que no son usuales de encontrar

Los puntos que se evalúan son los siguientes.

Documentación

Mínimamente debe explicar cómo correr el código.

Usabilidad

Son los aspectos que afectan cómo un usuario final utilizará el código. Por ejemplo, ¿la salida es consistente? (si ambas partes devuelven los resultados de la misma manera, ya sea consola o archivo) ¿puedo cambiar el nombre o cantidad de archivos a procesar (sin cambiar el código)?

Nota: La letra tiene un pequeño detalle, que la salida que se muestra no es un JSON válido (le faltan comas al final de línea). Si la prueba tiene esto en consideración, es una muestra de mucha atención al detalle.

Usabilidad Frontend

Son los aspectos que afectan cómo un usuario final utilizará la vista. Por ejemplo, ¿Es fiel al mock? ¿Los botones dicen lo que tiene que decir? ¿Se imprimen los usuarios? ¿Se puede navegar entre los módulos?

Calidad del código

¿Sigue buenas prácticas para asegurarse que el código es de buena calidad? Por ejemplo, ¿hay comentarios adecuados? ¿Evita repetición de código? ¿Tiene manejo de errores?

Calidad del código Frontend

¿Genera y utiliza componentes genéricos para el desarrollo de la solución? ¿Desacopla los estilos de CSS para que los estilos de diferentes componentes no se superpongan? ¿Mantiene una arquitectura clara de carpetas en el proyecto?

Eficacia y eficiencia

En la parte B esperamos que el algoritmo a utilizar sea eficaz (que el resultado contenga todos los módulos y que no arroje demasiados usuarios) más allá de cual sea el algoritmo elegido. Con los datos de prueba, la mejor solución requiere usar 4 usuarios. Consideramos aceptable devolver una solución de 5 usuarios.

Otros

Si bien dejamos documentada la evaluación en los puntos anteriores, para el proceso de selección lo más importante es la clasificación entre los 4 niveles (ver siguiente sección), y posible feedback que se pueda dar a la persona sobre la prueba. Dejamos constancia también de otras observaciones que no estén contempladas en el checklist.

En “aspectos que destacan (positivos o negativos), o impresiones generales”, intentamos poner lo que más nos llamó la atención. Indicamos con (-) y (+) si es positivo o negativo (ya que quién lee luego nuestros comentarios no necesariamente tiene formación técnica y a veces puede no quedar 100% claro si algo es bueno o malo).

En “otras notas de corrección” son otros comentarios que nos surgen durante la corrección, tal vez detallando más por qué algún punto se lo dimos negativo o positivo, o comentarios que no están contemplados por algún punto del checklist pero tiene sentido dejar aclarado.

El feedback que se provee debe ser algo que pueda ser “copiado y pegado” para entregar a la persona. Intentamos que sea alguna sugerencia que mejore la calidad de su código.

Niveles

Un resultado esperado es la categorización de la prueba técnica en niveles de acuerdo a lo que se esperaría de los niveles trainee, junior o semi senior. La prueba técnica no tiene suficiente complejidad como para juzgar mucho más que estos niveles y sólo mide un aspecto técnico (no se evalúan otras habilidades que se requieren en los distintos niveles). **La categorización es subjetiva**, pero damos las siguientes líneas generales con el objetivo de orientar cómo se clasificaría una prueba.

No suficiente

Es una persona que todavía está muy verde y no está preparada para afrontar este desafío. Le pasa alguna de:

- Comete errores graves
- No logra cumplir con lo pedido en la prueba
- Le lleva muchísimo tiempo (más de 12 horas para pruebas de backend y más de 24 horas para pruebas de frontend)

Frontend

- Toda la lógica dentro de un solo archivo.

Trainee

Es una persona que tiene potencial y conocimientos básicos, pero sería recomendable que pasara por la experiencia de ScaleUp.

- Realizar la prueba le lleva más de 6 horas para backend o más de 16 horas para frontend
- Tiene una documentación muy básica (“corra con python”) o inexistente.
- No tiene comentarios, o tiene comentarios de poca utilidad.
- Le faltan criterios de reutilización de código.
- El código es difícil de entender.

Frontend

- No mantiene una buena arquitectura de carpetas en el proyecto.
- Dejan CSS que no tiene sentido o que no aplica.

- No generan componentes genéricos ni reutilizan componentes.
- Lo realizado tiene fallas estéticas con respecto a lo requerido

Junior

Esperaríamos que esta persona fuera capaz de ingresar a un equipo y ser productiva en el corto plazo (1 mes).

- Realizar la prueba le lleva menos de 6 horas para backend o entre 8 y 16 horas para frontend
- Tiene una documentación adecuada, indica la línea de comandos a utilizar para correr la prueba, que versión de runtime se precisa y dónde ubicar los archivos de entrada en caso de querer cambiarlos.
- Organiza el código en funciones

Frontend

- Tiene una arquitectura básica y comprensible en el proyecto
- Demuestra al menos conocimiento básico de CSS
- Muestra intención de reutilizar componentes (aunque los componentes no sean los más eficientes)
- La web generada es similar o igual a la requerida en los diseños

Semi Senior

Esperaríamos que esta persona fuera capaz de ingresar a un equipo y ser productiva casi inmediatamente (1 semana).

- Realizar la prueba le lleva menos de 4 horas
- Tiene una documentación clara, que explica no sólo cómo utilizar el código sino también consideraciones que se tomaron en cuenta al realizar la solución (por ejemplo: por qué se eligió cierto algoritmo, que limitación puede tener el algoritmo)
- El código es consistente, fácil de leer, bien organizado, con comentarios adecuados (ojo que puede no tenerlos si el código es muy fácil de leer). Hace error handling (archivos de entrada en formato incorrecto).
- Posiblemente tiene algún test unitario.

Frontend

- Genera una arquitectura de carpetas claras. Por ejemplo: componentes, hooks, estilos, reducers, actions.
- Realiza componentes genéricos que se pueden reutilizar para wrappear el contenido.

- Utiliza herramientas avanzadas como el ruteo de React, Redux, styled-componentes, etc.
- El resultado generado es no sólo fiel al diseño requerido sino que le agrega detalles extras como:
    - Cambiar el cursor cuando se encuentra sobre botones
    - Los botones tienen un comportamiento diferente en los estados de hover o active
    - Utiliza alguna propiedad un poco más avanzada de CSS como puede ser animation o transition

Resultado esperado

Salida parte A


```
{
  "auth_module": {
    "authn.provider_1": [
      "./u17.json",
      "./u19.json",
      "./u3.json",
      "./u4.json"
    ],
    "authn.provider_2": [
      "./u1.json",
      "./u10.json",
      "./u13.json",
      "./u14.json",
      "./u16.json",
      "./u18.json",
      "./u6.json",
      "./u8.json"
    ],
    "authn.provider_3": [
      "./u0.json",
      "./u11.json",
      "./u12.json",
      "./u15.json",
      "./u7.json"
    ],
    "authn.provider_4": [
      "./u2.json",
```

```
      "./u5.json",
      "./u9.json"
    ]
  },
  "content_module": {
    "authz.provider_1": [
      "./u14.json",
      "./u4.json"
    ],
    "authz.provider_2": [
      "./u13.json",
      "./u15.json",
      "./u16.json",
      "./u17.json",
      "./u8.json",
      "./u9.json"
    ],
    "authz.provider_3": [
      "./u10.json",
      "./u11.json",
      "./u18.json",
      "./u2.json",
      "./u3.json",
      "./u5.json"
    ],
    "authz.provider_4": [
      "./u0.json",
      "./u1.json",
      "./u12.json",
      "./u19.json",
      "./u6.json",
      "./u7.json"
    ]
  }
}
```


Corrección parte B

Se debe verificar que los usuarios elegidos cubran todos los módulos

Está disponible verificador.py, que se corre en el directorio donde están los archivos json y se le ingresa el resultado (por ej: ['./u18.json', './u9.json', './u0.json', './u4.json']) y el programa indica si eso cubre todos los módulos o no.

Ejemplo

Readme

Me llevó 3 horas.

Instrucciones: Cambiar la variable 'path' en la función 'read_files' para apuntar a la carpeta donde estén los archivos json de ser necesario.

Correr el archivo.

Código

```
import os
import glob
import json
import itertools


def read_files(file_dictionary, part_a_dict, universe):
   path = './pruebatecnica/'
   for filename in glob.glob(os.path.join(path, '*.json')):
       # Se leen los archivos .json y se toman los datos necesarios
para procesamiento
       with open(os.path.join(os.getcwd(), filename), 'r') as file:
           data = json.load(file)
           short_filename = '.' + filename.split('./pruebatecnica')[-1]
           data_auth_module = data['provider']['auth_module']
           data_content_module = data['provider']['content_module']

           part_a(short_filename, part_a_dict, data_auth_module,
data_content_module)
           # Se forman las estructuras necesarias para la parte B
           file_dictionary[short_filename] = [
               data_auth_module, data_content_module
           ]
```

```
            universe.add(data_auth_module)
            universe.add(data_content_module)


def part_a(filename, part_a_dict, data_auth_module,
data_content_module):
    # Se agregan los modules al diccionario que recive como parámetro
    if 'auth_module' in part_a_dict:
        if data_auth_module in part_a_dict['auth_module']:

part_a_dict['auth_module'][data_auth_module].append(filename)
        else:
            part_a_dict['auth_module'][data_auth_module] = [filename]
    else:
        part_a_dict['auth_module'] = {data_auth_module: [filename]}

    if 'content_module' in part_a_dict:
        if data_content_module in part_a_dict['content_module']:

part_a_dict['content_module'][data_content_module].append(filename)
        else:
            part_a_dict['content_module'][data_content_module] =
[filename]
    else:
        part_a_dict['content_module'] = {data_content_module:
[filename]}


def part_b(sets, universe):
    number = 4
    while True and number < 10:
        # Se crean todas las combinaciones posibles para un cierto largo
        posibilities = [
            list(itertools.combinations(sets.keys(), number)),
            list(itertools.combinations(sets.values(), number))
        ]
        # Se iteran las combinaciones y de ser encontrada una que
contenga todos los elementos del universo se retorna
        # En caso contrario se aumenta el numero y se repite el proceso
hasta encontrar un caso positivo
```

```
        for i in range(len(posibilities[0])):
            # Se crea una lista con todos los elementos de la
combinación
            merged = list(itertools.chain(*posibilities[1][i]))
            # Se chequea si los elementos completan el universo
            found = all(elem in merged for elem in universe)
            if found:
                return list(posibilities[0][i])
        number += 1
    return "No combination found"


def main():
    file_dictionary = {}
    part_a_dict = {}
    universe = set()

    read_files(file_dictionary, part_a_dict, universe)
    result_b = part_b(file_dictionary, universe)

    print (f'Resultado parte A:\n{part_a_dict}')
    print (f'Resultado parte B:\n{result_b}')

if __name__ == '__main__':
    main()
```

Salida de ejecución

Resultado parte A:

{'auth_module': {'authn.provider_2': ['./u18.json', './u1.json', './u14.json', './u8.json', './u6.json', './u13.json', './u10.json', './u16.json'], 'authn.provider_4': ['./u5.json', './u9.json', './u2.json'], 'authn.provider_1': ['./u19.json', './u3.json', './u4.json', './u17.json'], 'authn.provider_3': ['./u11.json', './u0.json', './u12.json', './u7.json', './u15.json']}, 'content_module': {'authz.provider_3': ['./u18.json', './u5.json', './u3.json', './u11.json', './u2.json', './u10.json'], 'authz.provider_4': ['./u1.json', './u19.json', './u0.json', './u12.json', './u7.json', './u6.json'], 'authz.provider_1': ['./u14.json', './u4.json'], 'authz.provider_2': ['./u8.json', './u9.json', './u15.json', './u17.json', './u13.json', './u16.json']}}

Resultado parte B:

['./u18.json', './u9.json', './u0.json', './u4.json']

Corrección

Las salidas son correctas.

La columna de “comentarios explicativos” no es parte del checklist, se agrega aquí para explicar el razonamiento de por qué se marca de esa forma.

Resultado de la evaluación	Comentarios explicativos
📚 Documentación	
❌Explica cómo correr el código.	“Correr el código” no es explicación suficiente
❌Documenta la versión de la tecnología en la que programó la prueba.	
✅Explica cómo funciona el código, o elecciones que se tomaron, o condiciones particulares	Ya que indica cómo modificar el código para otra carpeta
👨🏻‍💻 Usabilidad	
✅El output es consistente (todo por consola o todo por archivos).	
✅Parametriza los archivos en la ejecución (archivo de config, o constante)	Ya que es relativamente sencillo cambiar de dónde levanta los archivos (es una variable, que si bien está en una función, está al principio del archivo)
✅No hardcodea el nombre de archivos (u*.json)	
✅No hardcodea la cantidad de archivos (20)	
✅No hardcodea los providers que espera encontrar (authx.provider_y)	
❌Imprime la salida como en la letra.	
🍝 Calidad del código	
✅Es consistente con la nomenclatura (camelCase o snake_case)	
✅Los comentarios son adecuados (dicen cosas útiles).	
✅No hay comentarios excesivos.	
✅Escribe de acuerdo a las convenciones de la tecnología.	
✅Divide el problema en funciones chicas y concretas.	
✅No repite el código de la parte A.	
❌❗ No hay código duplicado.	La función de la parte A duplica código (que podría ser un for, o una función auxiliar)

✅No tiene código mal indentado.	Esto en python es imposible :)
✅No tiene formato irregular (espacios o enters donde no debería).	
❌Tiene error handling.	
🛠️ Eficacia y Eficiencia	
✅Parte A correcta	
✅Consigue un set de usuarios que cubren todos los módulos.	
✅Busca un set reducido (ej: ordenar por los que usan más módulos).	
✅😀Asegura el set mínimo (ej: backtracking, generar todo, etc).	
🎚️ Nivel	
Junior	Recordar que esta categorización es subjetiva, se pueden tener todos los ticks y aún así no clasificar como Semi Senior.
⭐Aspectos que destacan (positivos o negativos), o impresiones generales	Qué fue lo más llamativo
(-) Es desprolijo por hardcodear límites a la búsqueda de solución	Obs: hay que tener cuidado que sea claro si esto es positivo o negativo, ya que quienes usan esto luego no necesariamente son técnicos
📝 Otras notas de corrección	
La documentación dice "correr el archivo", no es suficiente	
Indica como cambiar el path, pero no está en un lugar "fácil" del código	
En la parte A duplica el código para generar la salida	
En la parte B hardcodea la cantidad de cosas a generar, lo cual puede no generar una solución si hay más módulos	
Más allá de eso, demuestra conocimiento de Python (usa cosas "avanzadas" del lenguaje: itertools)	
Una variable está mal escrita (posibilities)	
Parece ser bueno técnicamente, pero desprolijo	
🎁 Feedback	
Tené cuidado con utilizar límites que dependen de los datos de	

entrada. Dejas fijo el máximo de combinaciones que probas.	
Sería más claro que utilizaras el valor de retorno de una función en vez de usar los parámetros como "in-out"	

Explicación puntos de corrección Frontend

Los siguientes son los puntos meramente de Frontend que se evalúan en la prueba técnica junto a un breve texto descriptivo.

👨🏻‍💻 Usabilidad Front	Descripción
Queda seleccionado el botón del módulo o submódulo	Los botones de módulo y submódulo quedan seleccionados luego de hacer click sobre ellos.
Los headers dicen lo que pide la letra	Los botones de módulo y submódulo tienen el texto que pide la letra.
Los botones dicen lo que pide la letra	Los botones de usuarios dicen lo que pide la letra.
Se utiliza el ruteo de react	Utiliza el ruteo de react y no uno custom.
Similitud visual del módulo a la letra	El componente de módulo que tiene borde + header + content se vea similar a lo pedido en la letra.
Similitud visual de los botones a la letra	Los botones de usuario, y más que nada los de “Create”, “Advice”, etc sean similares a la letra (importante ver los colores, iconos usados y posición de los íconos con respecto al texto).
Responsive	La solución generada es responsiva (si agrandamos o achicamos pantalla no se “rompe” y sigue siendo usable).
🍝 Calidad del código Front	
Crea componentes genéricos para el botón	Genera un componente genérico para el botón el cual puede utilizar tanto en los botones de módulo y submódulo como en los de usuario o de acciones (“create”, “advice”, etc).
Crea componentes genéricos para el Contenedor (header + content)	Genera un componente genérico que tiene dentro un header y un espacio para el “content” en el cual se puede rellenar con cualquier tipo de contenido dentro (no tiene hardcodeado otro contenedor o la parte de los usuarios).

Usa destructuring de las props	Desestructura las props en js. Ej: en lugar de recibir “functionName(props)” y usar la prop como “props.data”, se espera que reciba las props como “functionName({ data })” y usar la prop directamente como “data”.
Utiliza el redux store	Utiliza la Redux Store.
Genera otra carpeta para los componentes genéricos	Genera una carpeta en donde tenga todos los componentes genéricos (usualmente se la llama “components”).
El css está desacoplado	El CSS de cada componente NO afecta al CSS del resto de componentes. Esto usualmente se hace utilizando librerías como css-modules o styled-components, pero cualquier librería o técnica de desacople de estilos es válida para este punto.
Usa scss	Utiliza Sass o Scss.
Usa solo componentes funcionales o no funcionales (no los mezcla)	Utiliza un solo tipo de componente de react (funcional o de clases). No importa qué tipo de componente utilice, pero si importa que sean TODOS los componentes de un solo tipo.
Separa las constantes en archivos	Si tiene constantes en el código, estas deben estar separadas en un archivo separado. En el caso de que las constantes sean reutilizadas se deben colocar en un archivo que sea accesible a todos los componentes y no en la carpeta de un componente en específico (normalmente se coloca en “*src* > *constants”* o similar).
Arquitectura clara (CSS puesto junto al componente)	El proyecto debe tener una arquitectura de carpetas clara, los componentes deben permanecer junto a sus estilos, el proyecto debe tener carpetas como *“components”, “hooks”* (si los usa), *“constants”* (si las usa), etc. Y todo esto debe encontrarse dentro de una carpeta *“src”*. No tiene que ser la mejor arquitectura posible, pero sí una arquitectura en la que se comprenda qué cosa está en donde.

Pruebas a corregir

En la carpeta de Training de pruebas técnicas están disponibles 5 pruebas de backend y 3 pruebas de frontend que usaremos para calibrar. Se pide corregir esas pruebas y luego compararlas con las soluciones que están en el checklist de corrección (tabs Manual del 1 a 5 y Manual frontend del 1 a 3).

Se espera que los correctores hayan llegado a los mismos resultados en el checklist, y que estén de acuerdo en el nivel de la prueba. Las observaciones y el feedback son por su naturaleza, subjetivos.